



Tutorial:

Atividade 3 (Parte 2) - Evolução de Software

Equipe 4

Carlos Eduardo Dias dos Santos - 202100104941

Déborah Abreu Sales - 202100060758

Eduardo Afonso Passos Silva - 201800102096

Guilherme Ilan Barboza Carvalho - 201900051196

Marcelo Venicius Almeida Lima - 202000012981

Matheus Soares Santana - 201800147786

Mikael Douglas Santos Farias - 201700053275

Raí Rafael Santos Silva - 202000138043

COMP0441 - EVOLUÇÃO DE SOFTWARE (2025.2 - T01)
GLAUCO DE FIGUEIREDO CARNEIRO

Sumário

1. Contextualização.....	3
2. Melhoria 1 – Cache de Dependências no Pipeline.....	4
2.1 Problema Identificado.....	4
2.2 Solução Implementada.....	5
2.3 Evidência – Antes x Depois.....	7
3. Melhoria 2 – Documentação de CI/CD.....	7
3.1 Problema Identificado.....	7
3.2 Solução Implementada.....	8
4. Evidências da Execução.....	9
5. Reflexão – Impacto na “Coragem em Refatorar”.....	13
6. Contribuição dos Integrantes.....	14
7. Referências.....	14

1. Contextualização

A evolução de um software ao longo do tempo envolve a incorporação contínua de novas funcionalidades, correções de defeitos e refatorações estruturais necessárias para preservar sua qualidade interna. Entretanto, à medida que um projeto cresce em complexidade, torna-se inviável sustentar esse processo de forma segura sem o apoio de mecanismos automatizados capazes de validar alterações de maneira sistemática e reproduzível. Nesse contexto, práticas de Integração Contínua e Entrega Contínua (CI/CD) assumem um papel central na sustentação da evolução do software, atuando como barreiras técnicas contra regressões e como instrumentos de governança da qualidade.

Na Atividade 3 (Parte 1), a equipe realizou uma análise detalhada da infraestrutura de CI/CD do projeto open-source Mastra, mantido pela organização mastra-ai. A investigação teve como foco a identificação das ferramentas utilizadas, dos arquivos de configuração do pipeline e da estratégia de execução dos workflows automatizados. A partir dessa análise, foi possível constatar que o projeto já possui uma esteira de automação madura, baseada em GitHub Actions, com forte integração a ferramentas como PNPM, Turborepo, Vitest, ESLint e Changesets, além de múltiplos workflows especializados para diferentes domínios do sistema.

Com base nessas evidências, o projeto foi enquadrado no Cenário A – Projeto que já implementa CI/CD, caracterizado por um elevado nível de automação e por mecanismos rigorosos de validação antes da integração de código à branch principal. Esse cenário indica que o CI/CD no Mastra não atua apenas como um mecanismo operacional, mas como um elemento estruturante da evolução do software, influenciando práticas de refatoração, frequência de releases e integração de novos contribuidores.

Apesar do alto grau de maturidade identificado, a análise da Parte 1 também evidenciou limitações relevantes que impactam diretamente a experiência dos desenvolvedores e a eficiência do processo evolutivo. Entre essas limitações, destacam-se o tempo de feedback elevado observado em workflows mais complexos, que podem ultrapassar a marca de dez minutos, e a alta barreira de entrada para novos contribuidores, decorrente da complexidade do ecossistema de ferramentas e da ausência de documentação centralizada sobre a execução local do pipeline.

Diante desse diagnóstico, esta segunda parte da atividade tem como objetivo transcender a análise teórica e promover intervenções técnicas concretas na infraestrutura de CI/CD do projeto, atuando diretamente sobre os gargalos identificados. A proposta não é reformular integralmente o pipeline existente, mas realizar otimizações pontuais, coerentes com o nível de maturidade do projeto, capazes de melhorar o tempo de feedback e reduzir fricções no processo de contribuição.

Assim, a Atividade 3 (Parte 2) busca demonstrar, na prática, como pequenas melhorias na esteira de automação podem gerar impactos significativos na confiança do desenvolvedor, na coragem para refatorar e na sustentabilidade da evolução do software, reforçando o papel estratégico do CI/CD como facilitador, e não como obstáculo, do desenvolvimento contínuo.

2. Melhoria 1 – Cache de Dependências no Pipeline

O cache de dependências é uma técnica fundamental para otimização de pipelines de CI/CD, permitindo que pacotes e bibliotecas já baixados em execuções anteriores sejam reutilizados, evitando downloads redundantes e reduzindo significativamente o tempo de execução dos workflows.

No contexto do projeto Mastra, que utiliza pnpm como gerenciador de pacotes em um monorepo com mais de 800 dependências, a implementação adequada de cache torna-se ainda mais crítica para a eficiência do pipeline.

2.1 Problema Identificado

Ao analisar a infraestrutura de CI/CD do projeto Mastra, foram identificadas três principais deficiências relacionadas ao cache de dependências:

a) Cache Básico Limitado

A action centralizada `setup-pnpm-node` (localizada em `.github/actions/setup-pnpm-node/action.yaml`) utilizava apenas o parâmetro `cache: 'pnpm'` da action `actions/setup-node`. Esta configuração realiza cache apenas do arquivo `pnpm-lock.yaml` para determinar se houve mudanças nas dependências, mas não armazena em cache o diretório do pnpm store, onde os pacotes são efetivamente armazenados.

Configuração original:

```
- name: Setup Node.js 22.x
  uses: actions/setup-node@v6
  with:
    node-version: 22.13.0
    cache: 'pnpm'
```

Impacto: A cada execução do pipeline, mesmo quando as dependências não mudavam, os pacotes precisavam ser baixados novamente do registro npm, resultando em tempo adicional de 2 a 5 minutos por job.

b) Inconsistência Entre Workflows

Foram identificados três workflows que não utilizavam a action centralizada `setup-pnpm-node`, optando por configurar o pnpm e Node.js manualmente:

Workflow	Quantidade de Jobs Afetados
secrets.e2e.yml	8 jobs
npm-publish.yml	4 jobs
create-release.yml	2 jobs

Exemplo de configuração manual encontrada no arquivo **secrets.e2e.yml**:

```
- uses: pnpm/action-setup@v4
  name: Install pnpm
  with:
    run_install: false

- name: Setup Node.js 22.x
  uses: actions/setup-node@v5
  with:
    node-version: 22.13.0
    cache: 'pnpm'

- name: Install dependencies
  run: pnpm install
```

Impacto: Esta falta de padronização dificultava a manutenção do pipeline, pois qualquer melhoria implementada na action centralizada não era automaticamente aplicada a esses workflows, criando comportamento inconsistente e duplicação de código.

c) Ausência de Cache Explícito do pnpm Store

O projeto não utilizava a action **actions/cache** para armazenar explicitamente o diretório do pnpm store (onde os pacotes são fisicamente armazenados após o download). Esta é uma prática recomendada pela documentação oficial do pnpm para otimização em ambientes de CI/CD.

Impacto: Sem o cache do pnpm store, cada execução do comando pnpm install precisava:

1. Resolver todas as dependências a partir do lockfile
2. Verificar quais pacotes já existem no store (que estava vazio)
3. Baixar todos os pacotes novamente do registro npm
4. Extrair e vincular os pacotes ao projeto

Este processo repetitivo consumia recursos de rede e tempo de execução desnecessariamente, especialmente considerando que o projeto Mastra possui um grande número de dependências devido à sua arquitetura de monorepo.

2.2 Solução Implementada

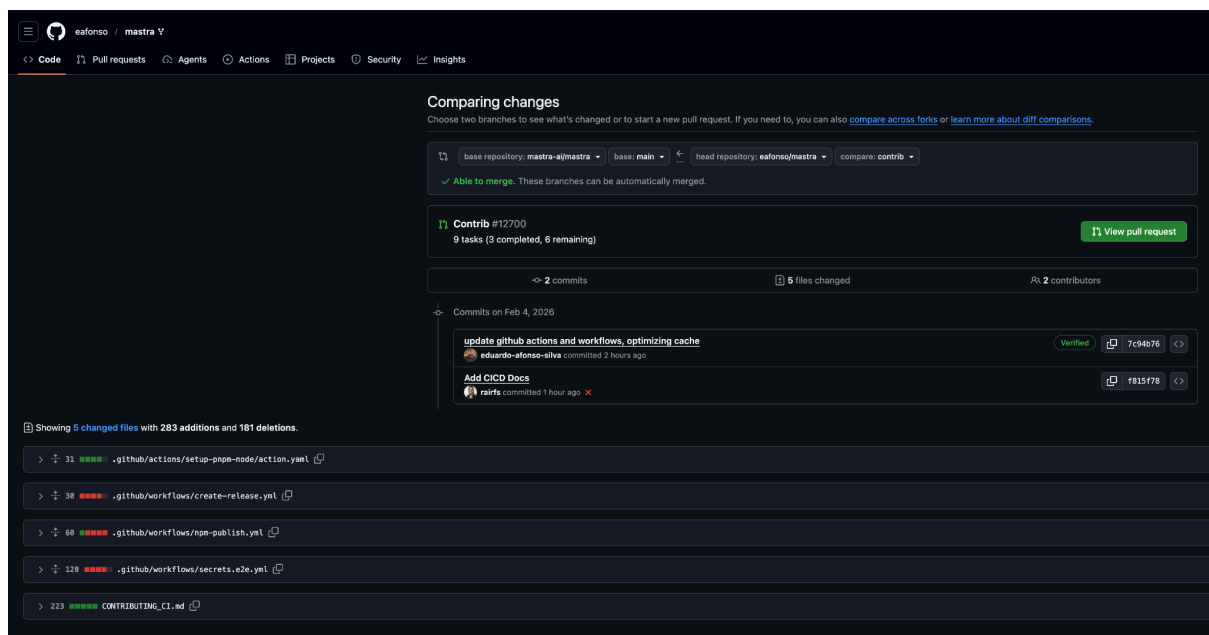
Descrição da solução implementada: Refatoração dos Workflows de GitHub Actions

A solução implementada no fork do projeto consistiu em uma refatoração da infraestrutura de Integração Contínua (CI) baseada em GitHub Actions, com o objetivo de reduzir duplicação de configuração, melhorar a manutenibilidade dos workflows e padronizar o ambiente de execução utilizado nos pipelines automatizados.

Antes da refatoração, os workflows continham configurações repetidas para a preparação do ambiente de execução, como a definição da versão do Node.js, a instalação do gerenciador de pacotes **pnpm** e a configuração de cache de dependências. Essa repetição aumentava o custo de manutenção, pois qualquer alteração nesses passos exigia modificações em múltiplos arquivos de workflow.

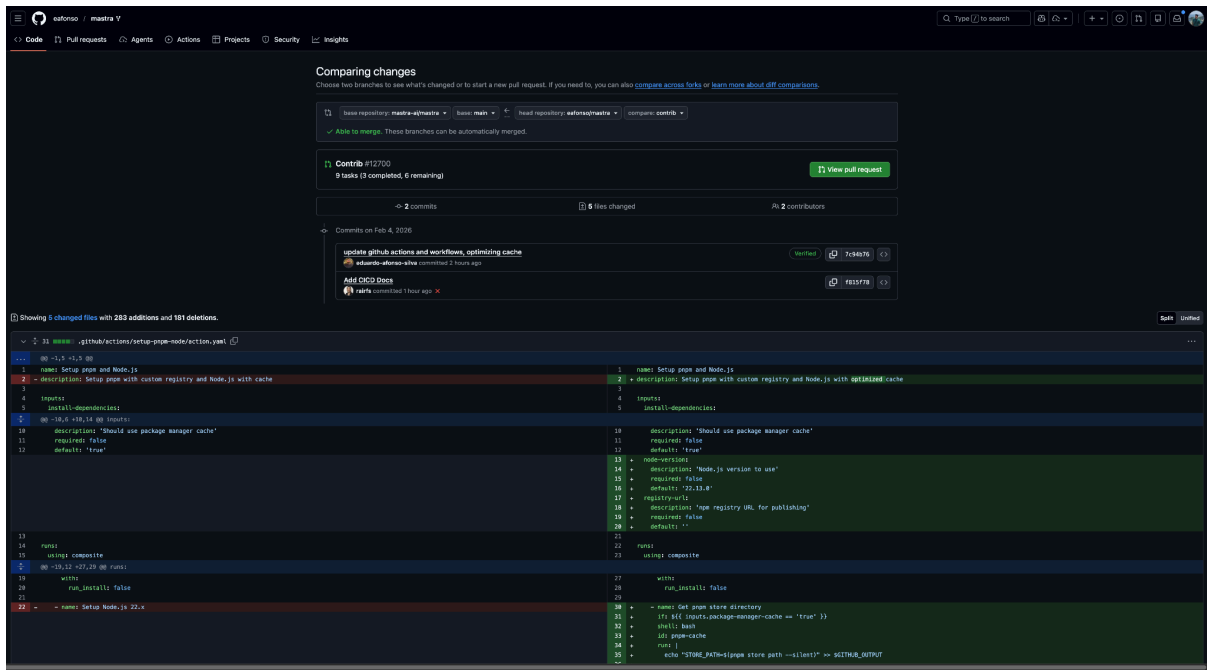
Como solução, foi criada uma GitHub Action customizada reutilizável, localizada no diretório **.github/actions/**, responsável por centralizar o processo de configuração do ambiente. Essa ação encapsula etapas comuns, incluindo:

- Definição e instalação da versão do Node.js;
- Configuração do **pnpm** como gerenciador de dependências;
- Ativação e reutilização de cache de dependências para otimização do tempo de execução dos pipelines.



Após a criação da action reutilizável, os workflows existentes foram ajustados para consumir essa ação, substituindo blocos redundantes de configuração por uma única chamada padronizada. Essa abordagem promoveu uma arquitetura mais modular, alinhada aos princípios de reutilização e separação de responsabilidades.

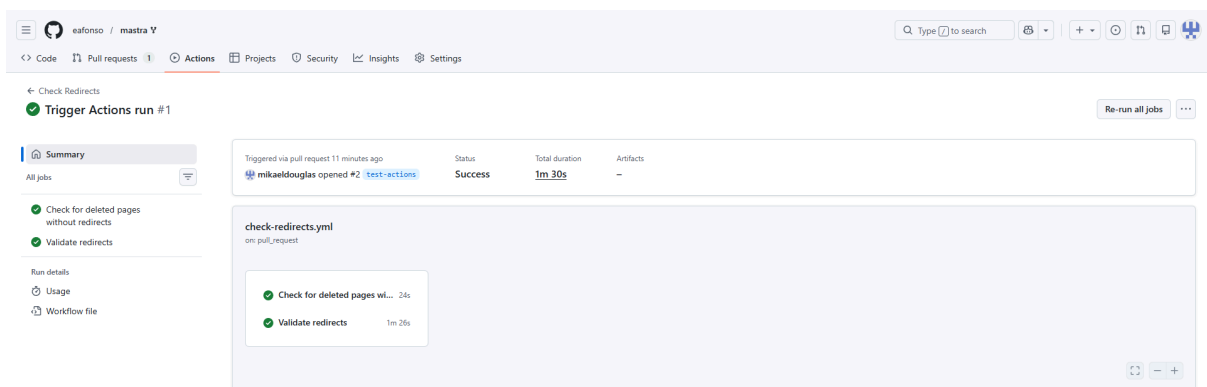
Mais detalhes da implementação podem ser visualizados no fork criado <https://github.com/eafonso/mastra/tree/contrib>.



2.3 Evidência – Antes x Depois

Para evidenciar a execução e a consistência do pipeline após as alterações no fork, comparamos duas execuções consecutivas do workflow Check Redirects acionadas pelo Pull Request da equipe (run #1 e run #2). Em ambas as execuções, o workflow concluiu com status Success e duração total de 1m30s, demonstrando que a esteira permanece estável e reproduzível entre execuções subsequentes. Como este workflow é voltado a validações de redirecionamento/documentação e não envolve instalação de dependências, ele foi utilizado aqui como evidência objetiva de execução do pipeline e como referência de tempo para uma etapa específica do CI.

Antes (run #1): 1m30s (print)



Depois (run #2): 1m30s (print).

The screenshot displays the GitHub Actions interface for a repository named 'eafonso / mastra V'. The 'Actions' tab is selected, showing a workflow named 'Check Redirs' with a status of 'Success'. The workflow was triggered via a pull request 2 minutes ago and completed in 1m 30s. The workflow file 'check-redirs.yml' is shown, containing two jobs: 'Check for deleted pages without redirects' and 'Validate redirects', both of which completed successfully. The interface includes a sidebar with navigation options like 'Summary', 'Run details', and 'Usage', and a top navigation bar with links to 'Code', 'Pull requests', 'Actions', 'Projects', 'Security', 'Insights', and 'Settings'.

3. Melhoria 2 – Documentação de CI/CD

A documentação de CI/CD de um projeto descreve como o processo de integração contínua está configurado

No contexto do repositório analisado observou-se que, não há documentação explícita explicando o funcionamento do CI/CD, ou seja, não existe orientação clara no README.md ou CONTRIBUTING.md

Embora o repositório contenha código de workflows de CI/CD dentro da pasta **.github/workflows** (o que demonstra o uso do GitHub Actions para automação), essa informação está dispersa nos arquivos de configuração e não é documentada de forma descritiva para potenciais contribuidores.

Isso significa que um desenvolvedor que deseja contribuir pode não compreender facilmente o que o pipeline executa nem como testar corretamente antes de submeter pull requests, o que resulta em maior dependência de tentativa e erro no ambiente remoto de CI e um aumento na dificuldade de contribuir para o projeto.

3.1 Problema Identificado

O arquivo principal de documentação do projeto, **README.md**, contém informações sobre a finalidade do framework, suas funcionalidades principais e como iniciar o uso básico da ferramenta, mas não apresenta seções que expliquem o pipeline automatizado de CI/CD

ou instruções sobre como reproduzir localmente as etapas executadas nos workflows automatizados.

Embora exista um arquivo **CONTRIBUTING.md** no repositório, este não inclui instruções específicas relacionadas à configuração ou execução do pipeline de CI/CD, nem orienta sobre como rodar build, testes ou validações localmente antes de submeter contributions ou pull requests.

Vale ressaltar que, o projeto possui diversos workflows configurados no GitHub Actions, identificáveis pela aba “**Actions**” do repositório, que mostram várias pipelines (como testes de pacotes, linting e validações) sendo executadas automaticamente em cada push ou pull request. Contudo, **essas configurações estão apenas em arquivos de workflow YAML e não são documentadas de forma descritiva ou consolidada em nenhum dos principais documentos do repositório.**

Outro ponto que evidencia o problema é o fato de que no guia de desenvolvimento interno (DEVELOPMENT.md), embora haja instruções para construir e testar o projeto usando Vite e comandos como `pnpm test`, essas instruções não são referenciadas ou explicadas como parte do pipeline de CI/CD do projeto, nem são vinculadas diretamente à experiência do contribuinte que deseja reproduzir a pipeline localmente

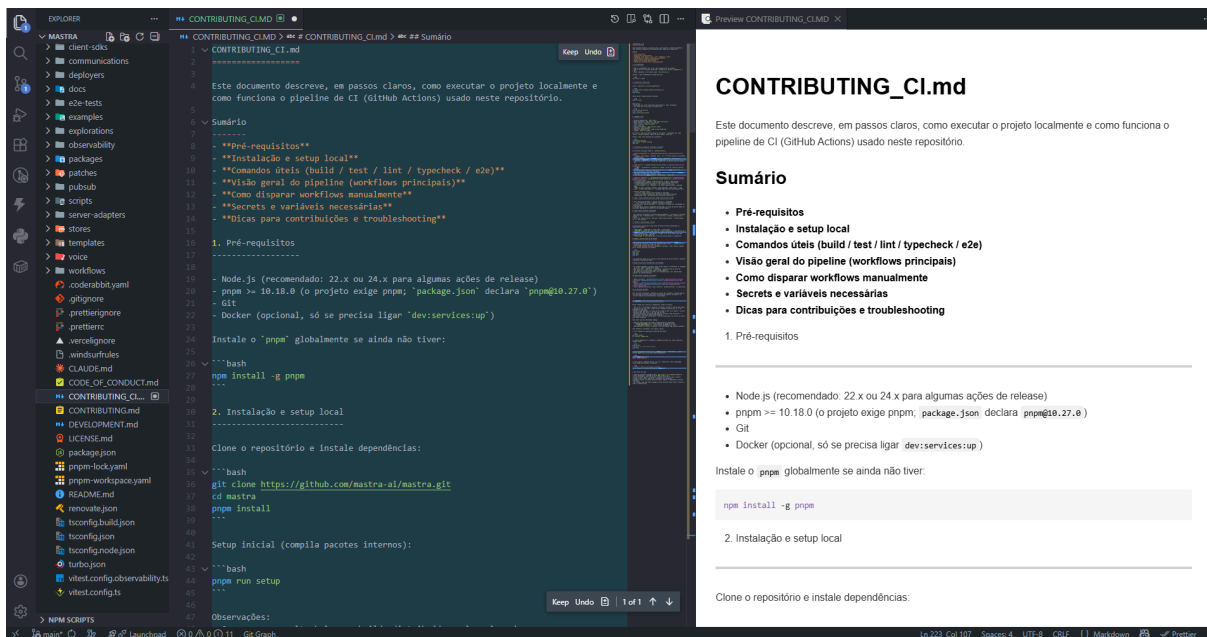
Sendo assim, perguntas como...

- **Quais comandos internos os workflows automatizados do GitHub Actions executam?**
- **Como reproduzir localmente as etapas de build e testes?**
- **Como depurar falhas que ocorrem na pipeline de CI?**

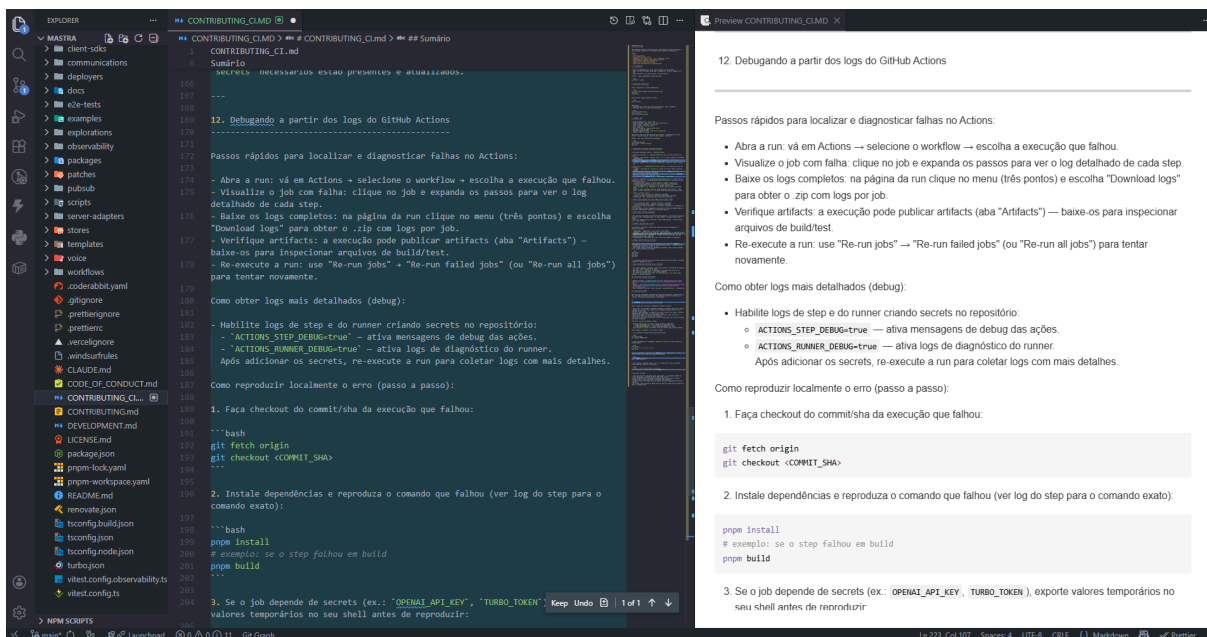
Não são respondidas em nenhuma documentação do projeto

3.2 Solução Implementada

Atualmente, o projeto Mastra possui uma arquitetura de CI/CD sofisticada, integrando Turborepo, pnpm e Changesets para garantir performance e entregas contínuas. No entanto, não havia até o momento uma documentação centralizada que explicasse o funcionamento desses fluxos ou como executá-los localmente, conforme já explicado na seção anterior. Sendo assim, foi desenvolvido um arquivo no formato .md que possui todas as instruções dos pipelines de CI/CD do projeto Mastra. Um fork foi realizado do repositório inicial e neste novo repositório (<https://github.com/eafonso/mastra>) foi criado o arquivo CONTRIBUTING_CI.md, conforme na imagem abaixo.



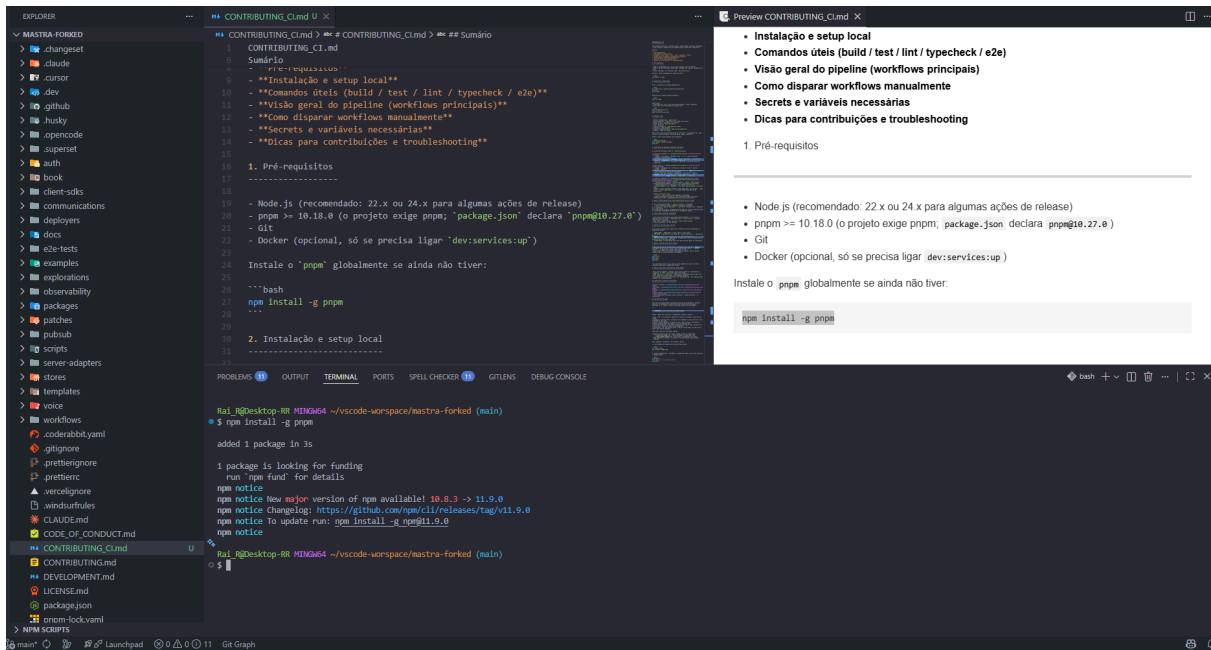
Todo o arquivo busca responder as perguntas levantadas na seção anterior deste documento, afinal o objetivo deste documento é diminuir a barreira inicial para novos desenvolvedores. Sendo assim, este arquivo cita, por exemplo, como depurar falhas que ocorrem na pipeline de CI. Este passo-a-passo pode ser encontrado ao final do arquivo .md que foi criado, conforme a imagem a seguir.



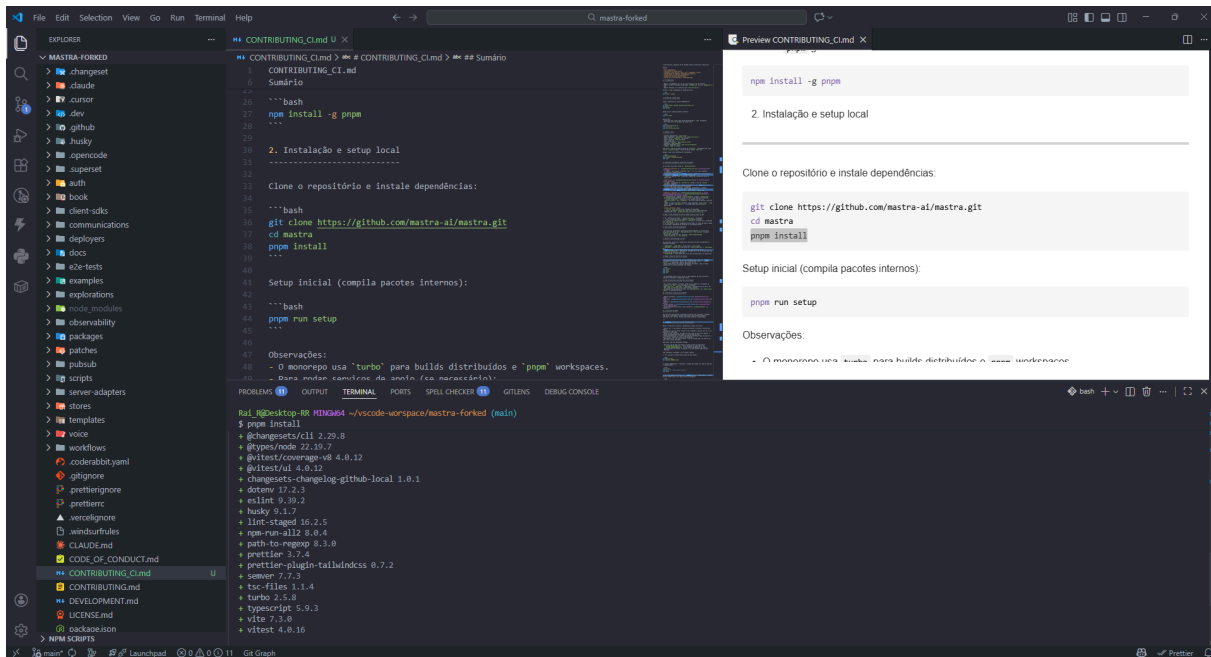
4. Evidências da Execução

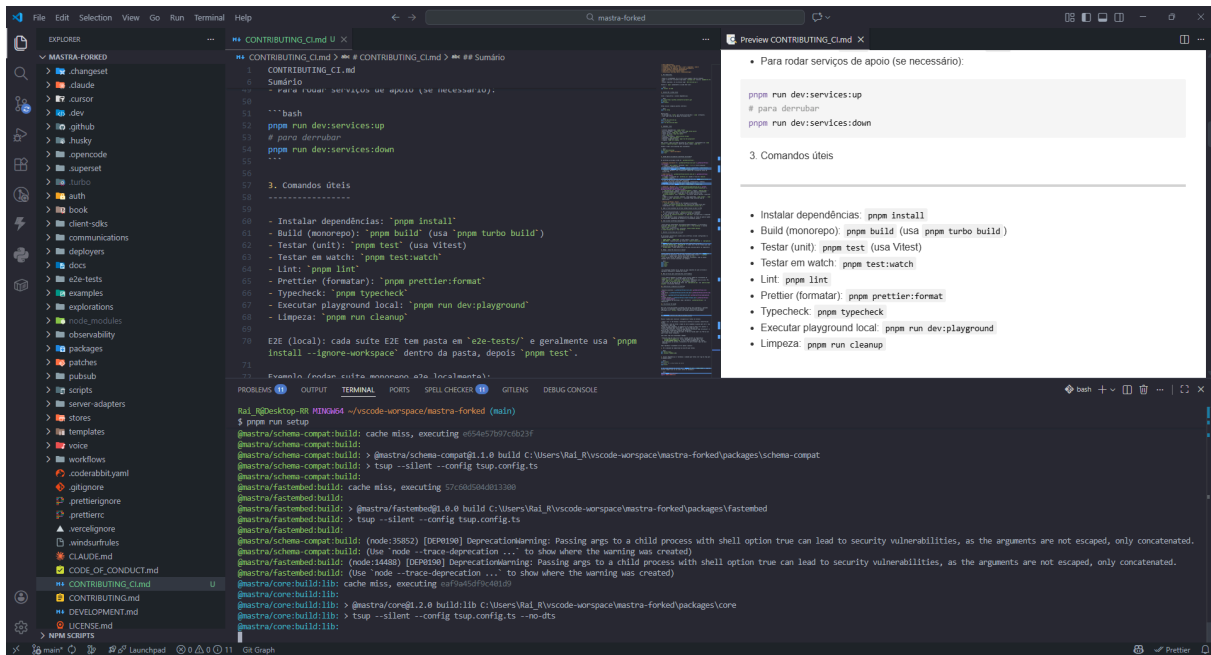
A seguir será demonstrado de forma aplicada a execução local do fluxo de CI/CD no projeto fork do mastra-ai.

Passo 1: Instalação das dependências



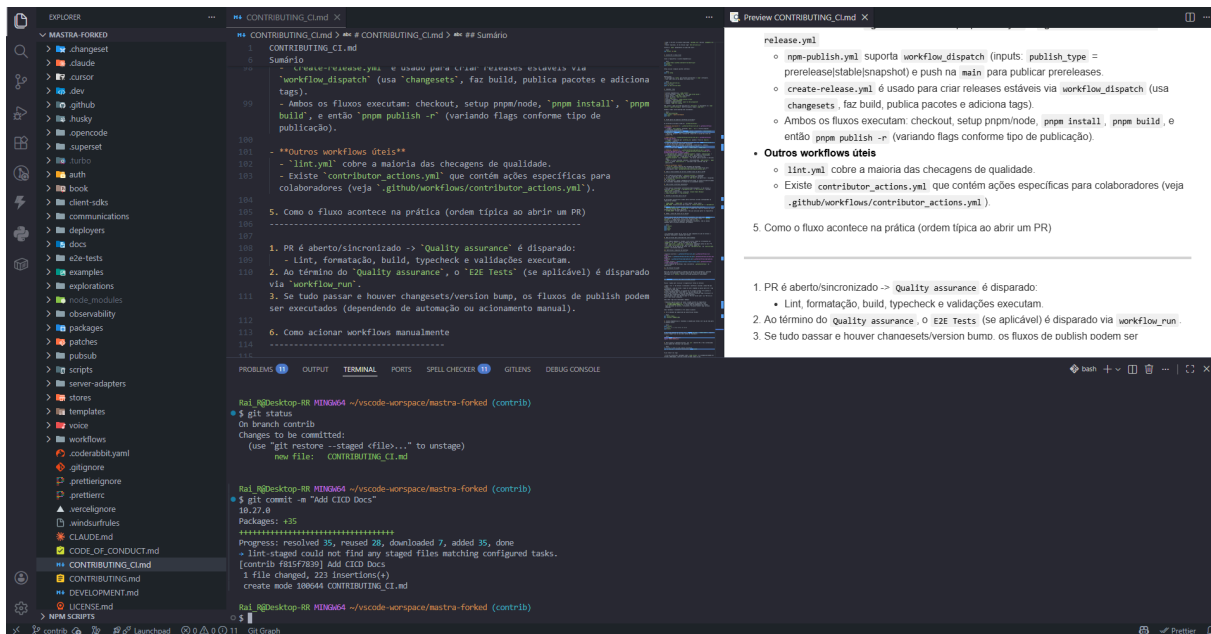
Passo 2: Configuração local



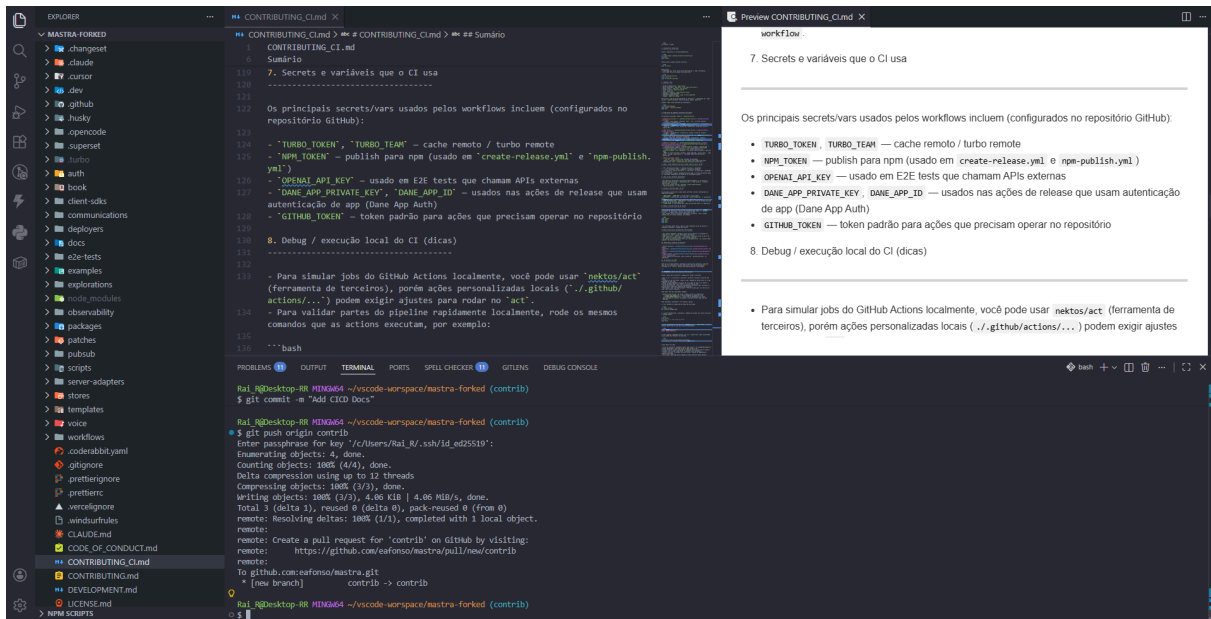


Passo 3: Alteração na base de código

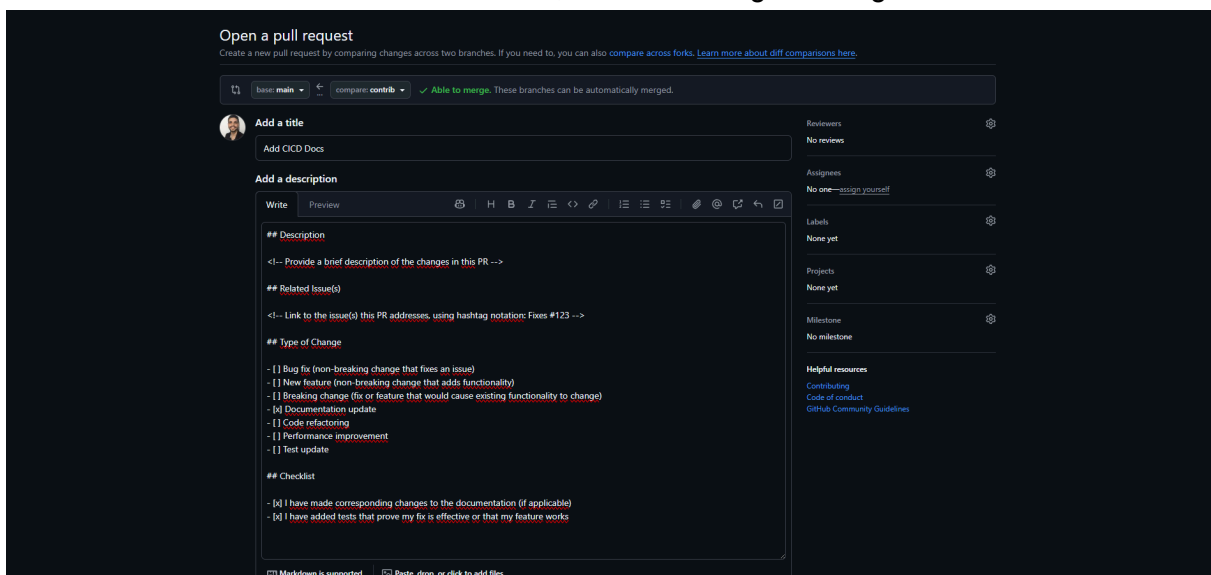
Após alteração necessária, basta realizar criar nova branch e realizar o commit. O pnpm vai cuidar de toda configuração, conforme imagem abaixo.



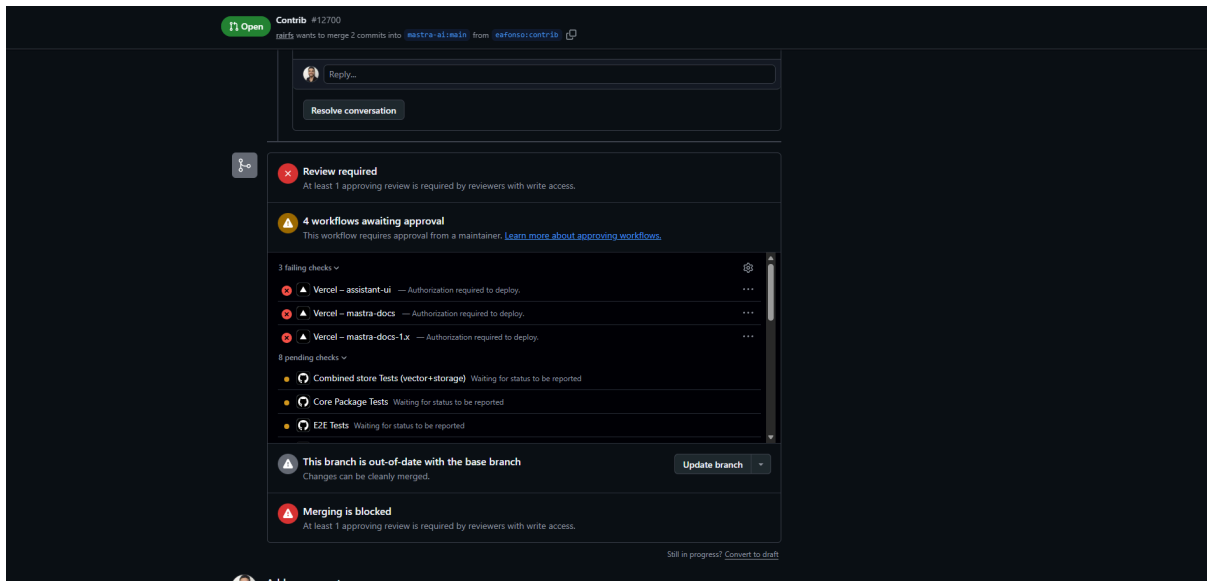
Então cria a nova branch remota, conforme imagem abaixo.



Cria PR no GitHub. Um ponto de observação é que o texto do PR é preenchido de forma automática e funciona como um formulário, conforme imagem a seguir.



No projeto original é necessário a aprovação dos desenvolvedores do projeto para que seja realizado o PR com sucesso, conforme imagem abaixo.



Após o PR aprovado, todo o processo que já foi explicado no arquivo CONTRIBUTING_CI.md será executado.

5. Reflexão – Impacto na “Coragem em Refatorar”

A partir das melhorias implementadas nesta atividade, é possível refletir sobre o impacto direto que a infraestrutura de CI/CD exerce na confiança dos desenvolvedores ao realizar mudanças no código, especialmente refatorações. Conforme discutido na contextualização, o projeto Mastra já possui um pipeline de CI/CD maduro e rigoroso, que atua como um elemento estruturante da evolução do software. No entanto, também foram identificados gargalos relacionados ao tempo de feedback e à complexidade do processo de contribuição.

A implementação do cache de dependências, descrita na Melhoria 1, atua diretamente na redução do tempo de execução dos workflows, um problema previamente identificado na análise do pipeline. Ao diminuir o tempo necessário para validar alterações, o desenvolvedor passa a receber respostas mais rápidas sobre o impacto de suas mudanças. Esse feedback ágil reduz a fricção associada a alterações estruturais no código, tornando o processo de refatoração menos custoso e mais previsível dentro do fluxo de desenvolvimento.

De forma complementar, a Melhoria 2, voltada à documentação de CI/CD, contribui para a redução da incerteza enfrentada por novos contribuidores. Conforme destacado anteriormente, a ausência de documentação centralizada dificultava a compreensão das etapas executadas nos workflows e aumentava a dependência de tentativas e erros no ambiente remoto de CI. Com a criação do arquivo CONTRIBUTING_CI.md, o processo passa a ser mais transparente, permitindo que desenvolvedores compreendam previamente quais validações serão executadas e como reproduzi-las localmente.

Essas duas melhorias, quando analisadas em conjunto, fortalecem a confiança no pipeline automatizado do projeto. Um ambiente de CI/CD mais rápido e melhor

documentado reduz o receio de introduzir mudanças mais profundas no código, uma vez que os mecanismos de validação tornam-se mais acessíveis, previsíveis e eficientes. Assim, a refatoração deixa de ser percebida como uma atividade arriscada e passa a ser integrada de forma natural ao processo de evolução do software.

6. Contribuição dos Integrantes

Participante	Matrícula	Contribuição
Carlos Eduardo Dias dos Santos	202100104941	Melhoria 1 - Descrição e implementação da solução.
Déborah Abreu Sales	202100060758	Contextualização da atividade.
Eduardo Afonso Passos Silva	201800102096	Melhoria 1 - Cache de Dependências. Descrição do que é o cache de dependências e descrição dos problemas identificados.
Guilherme Ilan Barboza Carvalho	201900051196	Análise de melhorias, mensurar impactos e possíveis vantagens das melhorias solicitadas.
Marcelo Venicius Almeida Lima	202000012981	Melhoria 2 - Documentação CI/CD. Problema identificado.
Matheus Soares Santana	201800147786	Não contribuiu.
Mikael Douglas Santos Farias	201700053275	Melhoria 1 - Evidência - Antes x Depois.
Raí Rafael Santos Silva	202000138043	Melhoria 2 - Execução da melhoria.

7. Referências

MASTRA. **AI framework for building agents and applications**. Disponível em: <https://github.com/mastra-ai/mastra>. Acesso em: 04 fev. 2026.

OPENAI. **ChatGPT**. Modelo de linguagem baseado em inteligência artificial. Disponível em: <https://chat.openai.com>. Acesso em: 04 fev. 2026.

Link do Vídeo de Relato:

 Carlos_Santos_Deborah_Sales_Eduardo_Silva_Guilherme_Carvalho_Marcelo...

GitHub da Atividade:

https://github.com/rairfs/Evolucao_Software_2025-2_mastra-ai_atividade_3_parte2